

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

NAME: A. Suraj Varma

Reg.No: 192525339

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Case Study (Medium)

Assessment Tool Weightage: 30%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	5
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	5
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	5
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	5
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	5
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	5

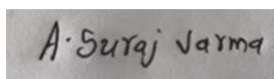
7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	5
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	5
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	5
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	5

Code of Conduct:

*I **A. Suraj varma (192525339)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning

Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q1. Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.

Answer:

Given relation:

ORDER(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

The natural functional dependencies are:

OrderID $\rightarrow$ CustID, CustName

ProdID $\rightarrow$ ProdName, Price

(OrderID, ProdID) $\rightarrow$ Qty

This means OrderID identifies the customer, ProdID identifies the product details, and the combination of OrderID and ProdID identifies the quantity ordered.

The candidate key is:

(OrderID, ProdID)

because both OrderID and ProdID are required to determine all attributes.

The original relation contains partial dependencies:

OrderID $\rightarrow$ CustID, CustName

ProdID $\rightarrow$ ProdName, Price

These attributes depend only on part of the composite key. Therefore, the relation is not in 2NF.

2NF Decomposition:

ORDER_DETAIL(OrderID, ProdID, Qty)

CUSTOMER(CustID, CustName)

PRODUCT(ProdID, ProdName, Price)

However, OrderID must identify the customer. Therefore, a more complete 3NF design is:

ORDER(OrderID, CustID)

CUSTOMER(CustID, CustName)

PRODUCT(ProdID, ProdName, Price)

ORDER_DETAIL(OrderID, ProdID, Qty)

Keys:

ORDER:

OrderID – Primary Key

CUSTOMER:

CustID – Primary Key

PRODUCT:

ProdID – Primary Key

ORDER_DETAIL:

(OrderID, ProdID) – Composite Primary Key

CustID in ORDER and OrderID/ProdID in ORDER_DETAIL are foreign keys.

Conclusion:

The original relation is decomposed into CUSTOMER, ORDER, PRODUCT and ORDER_DETAIL. This removes partial dependencies and redundancy and produces a 3NF design with better data integrity.

Q2. A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.

Answer:

Given relation:

PATIENT(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

The functional dependencies are:

$PID \rightarrow PName, DocID, WardNo$

$DocID \rightarrow DocName, Specialization$

$WardNo \rightarrow WardName$

The candidate key is PID because PID determines all the other attributes.

There are transitive dependencies:

$PID \rightarrow DocID \rightarrow DocName, Specialization$

and

$PID \rightarrow WardNo \rightarrow WardName$

These dependencies cause redundancy.

Anomalies:

1. Update Anomaly:

If a doctor's specialization changes, it must be updated for every patient treated by that doctor.

2. Insertion Anomaly:

A new doctor cannot be stored unless a patient is also associated with the doctor.

3. Deletion Anomaly:

If the last patient of a doctor is deleted, information about that doctor may also be lost.

BCNF Decomposition:

PATIENT(PID, PName, DocID, WardNo)

DOCTOR(DocID, DocName, Specialization)

WARD(WardNo, WardName)

Keys:

PATIENT:

PID – Primary Key

DOCTOR:

DocID – Primary Key

WARD:

WardNo – Primary Key

DocID and WardNo in PATIENT are foreign keys.

BCNF Verification:

In DOCTOR:

$DocID \rightarrow DocName, Specialization$

DocID is the key, so the relation satisfies BCNF.

In WARD:

$WardNo \rightarrow WardName$

WardNo is the key, so the relation satisfies BCNF.

In PATIENT:

$PID \rightarrow PName, DocID, WardNo$

PID is the key, so the relation satisfies BCNF.

Conclusion:

The original hospital relation is decomposed into PATIENT, DOCTOR and WARD. The transitive dependencies and redundancy are removed, and the resulting relations satisfy BCNF.

Q3. Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.

Answer:

Given relation:

ENROLLMENT(SID, SName, CourseID, CourseName, Faculty, Grade)

The functional dependencies are:

$SID \rightarrow SName$

$CourseID \rightarrow CourseName, Faculty$

$(SID, CourseID) \rightarrow Grade$

The candidate key is:

$(SID, CourseID)$

because a student can enroll in many courses and a course can have many students.

Step 1: Identify Partial Dependencies

SName depends only on SID:

$SID \rightarrow SName$

CourseName and Faculty depend only on CourseID:

$CourseID \rightarrow CourseName, Faculty$

These are partial dependencies because SID and CourseID are parts of the composite candidate key.

Therefore, the relation is not in 2NF.

Step 2: Decompose the Relation

STUDENT:

STUDENT(SID, SName)

COURSE:

COURSE(CourseID, CourseName, Faculty)

ENROLLMENT:

ENROLLMENT(SID, CourseID, Grade)

Keys:

STUDENT:

SID – Primary Key

COURSE:

CourseID – Primary Key

ENROLLMENT:

$(SID, CourseID)$ – Composite Primary Key

SID and CourseID in ENROLLMENT are foreign keys.

Conclusion:

The decomposition removes all partial dependencies. Each non-key attribute depends on the complete key of its relation. Therefore, the resulting design satisfies 2NF and reduces redundancy.

Q4. Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.**Answer:**

Given relation:

BOOKING(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

The following functional dependencies can be identified:

PassID $\rightarrow$ PassName

FlightNo $\rightarrow$ DeptCity, ArrCity

A passenger's seat is identified for a particular flight:

(FlightNo, PassID) $\rightarrow$ Seat, Date

The candidate key is:

(FlightNo, PassID)

because both the flight and passenger are required to identify a booking.

Step 1: Remove Partial Dependencies

PassID $\rightarrow$ PassName

FlightNo $\rightarrow$ DeptCity, ArrCity

These depend only on parts of the composite key.

Therefore, decompose into:

PASSENGER(PassID, PassName)

FLIGHT(FlightNo, DeptCity, ArrCity)

BOOKING(FlightNo, PassID, Seat, Date)

Step 2: Check BCNF

PASSENGER:

PassID $\rightarrow$ PassName

PassID is the key, so it is in BCNF.

FLIGHT:

FlightNo $\rightarrow$ DeptCity, ArrCity

FlightNo is the key, so it is in BCNF.

BOOKING:

(FlightNo, PassID) $\rightarrow$ Seat, Date

(FlightNo, PassID) is the key, so it is in BCNF.

Final BCNF Schema:

PASSENGER(PassID, PassName)

FLIGHT(FlightNo, DeptCity, ArrCity)

BOOKING(FlightNo, PassID, Seat, Date)

Conclusion:

The original relation contains partial dependencies caused by the composite key. After decomposition, each determinant is a candidate key of its relation. Therefore, the final design satisfies BCNF.

Q5. Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.**Answer:**

Since the question does not provide a complete HR relation, consider the following representative schema:

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, Location)

The functional dependencies are:

$\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}$

$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}, \text{Location}$

$\text{ManagerID} \rightarrow \text{ManagerName}$

Therefore, the following transitive dependencies exist:

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}$

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{ManagerID}$

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{Location}$

and:

$\text{EmpID} \rightarrow \text{ManagerID} \rightarrow \text{ManagerName}$

These transitive dependencies create redundancy.

3NF Decomposition:

EMPLOYEE(EmpID, EmpName, DeptID)

DEPARTMENT(DeptID, DeptName, ManagerID, Location)

MANAGER(ManagerID, ManagerName)

Keys:

EMPLOYEE:

EmpID – Primary Key

DEPARTMENT:

DeptID – Primary Key

MANAGER:

ManagerID – Primary Key

DeptID in EMPLOYEE is a foreign key.

ManagerID in DEPARTMENT is a foreign key.

3NF Verification:

In EMPLOYEE:

$\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}$

EmpID is the key.

In DEPARTMENT:

$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}, \text{Location}$

DeptID is the key.

In MANAGER:

ManagerID $\rightarrow$ ManagerName

ManagerID is the key.

Therefore, there are no transitive dependencies among non-key attributes.

Conclusion:

The HR schema is normalized to 3NF by separating employee, department and manager information. This reduces redundancy and prevents insertion, deletion and update anomalies.

Q6. Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.

Answer:

Given relation:

LIBRARY(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

Assume that a member can borrow multiple books and that a book can have multiple independent authors/genres where applicable.

The main multi-valued dependency caused by independent book-related information can be represented as:

MemberID $\twoheadrightarrow$ BookID

MemberID $\twoheadrightarrow$ BorrowDate

MemberID $\rightarrow$ MemberName

BookID $\rightarrow$ Title, Author, Genre

The problem is that multiple independent values associated with the same member can create unnecessary combinations.

4NF Decomposition:

MEMBER:

MEMBER(MemberID, MemberName)

BOOK:

BOOK(BookID, Title, Author, Genre)

BORROW:

BORROW(MemberID, BookID, BorrowDate)

Keys:

MEMBER:

MemberID – Primary Key

BOOK:

BookID – Primary Key

BORROW:

(MemberID, BookID, BorrowDate) can identify a borrowing event if the same book can be borrowed multiple times.

Explanation:

Member details are stored separately from book details. The borrowing relationship is stored in a separate relation.

This prevents unnecessary repetition of member and book information.

Conclusion:

The independent multi-valued information is separated into MEMBER, BOOK and BORROW relations. This removes multi-valued redundancy and provides a 4NF design.

Q7. Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.

Answer:

Since the question does not specify the complete telecom schema, consider the following representative relation:

BILLING(CustomerID, PhoneNo, CustomerName, PlanID, PlanName, CallID, CallDuration, Charge)

The functional dependencies are:

CustomerID $\rightarrow$ CustomerName

PhoneNo $\rightarrow$ CustomerID, CustomerName

PlanID $\rightarrow$ PlanName

CallID $\rightarrow$ PhoneNo, CallDuration, Charge

A suitable candidate key for the complete billing record is:

(CallID)

because CallID uniquely identifies a call record and determines the other call-specific information.

If the design allows multiple billing records per call period, a composite key such as:

(PhoneNo, CallID)

may also be considered depending on the business rules.

Candidate Keys:

CallID

or, if CallID is not globally unique:

(PhoneNo, CallID)

Primary Key:

CallID can be selected as the primary key when it is unique for every call.

Functional Dependencies:

CustomerID $\rightarrow$ CustomerName

PhoneNo $\rightarrow$ CustomerID

PlanID $\rightarrow$ PlanName

CallID $\rightarrow$ PhoneNo, CallDuration, Charge

Transitive Dependency:

CallID $\rightarrow$ PhoneNo $\rightarrow$ CustomerID $\rightarrow$ CustomerName

This shows that customer information should not be repeatedly stored in every billing record.

Conclusion:

Identifying candidate keys and functional dependencies before normalization helps determine the correct primary keys and reveals redundancy. The schema can then be decomposed into

CUSTOMER, PLAN, PHONE/CUSTOMER and CALL/BILLING relations for further normalization.

Q8. A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.

Answer:

Given relation:

PRODUCT(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

Assume the following functional dependencies:

$PID \rightarrow PName, SupplierID, Category, Price, Warehouse$

$SupplierID \rightarrow SupplierName$

Therefore:

$PID \rightarrow SupplierID \rightarrow SupplierName$

This is a transitive dependency.

Step 1: 1NF

All attributes contain atomic values, so the relation is in 1NF.

Step 2: 2NF

The candidate key is PID, which is a single attribute. Therefore, there is no partial dependency and the relation is in 2NF.

Step 3: 3NF

SupplierName depends on SupplierID rather than directly on PID.

Therefore, decompose into:

PRODUCT(PID, PName, SupplierID, Category, Price, Warehouse)

SUPPLIER(SupplierID, SupplierName)

Step 4: BCNF

In PRODUCT:

$PID \rightarrow PName, SupplierID, Category, Price, Warehouse$

PID is the candidate key.

In SUPPLIER:

$SupplierID \rightarrow SupplierName$

SupplierID is the candidate key.

Therefore, both relations satisfy BCNF under the given dependencies.

Final Normalized Design:

PRODUCT:

PID – Primary Key

PName

SupplierID – Foreign Key

Category

Price

Warehouse

SUPPLIER:

SupplierID – Primary Key

SupplierName

Conclusion:

The original relation is normalized from 1NF through 2NF and 3NF. The transitive dependency between SupplierID and SupplierName is removed, resulting in a clean BCNF design with reduced redundancy and improved data integrity.

Q9. Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.

Answer:

Since the question does not provide the detailed School ERP schema, consider the following representative relation:

SCHOOL(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Grade)

The functional dependencies are:

StudentID $\rightarrow$ StudentName, ClassID

ClassID $\rightarrow$ ClassName, TeacherID

TeacherID $\rightarrow$ TeacherName

SubjectID $\rightarrow$ SubjectName

(StudentID, SubjectID) $\rightarrow$ Grade

Problems in the Original Schema:

1. Student details are repeated for every subject.
2. Class information is repeated for every student.
3. Teacher information is repeated for every class record.
4. Subject information is repeated for every student-subject combination.

Anomalies:

Update anomaly:

Changing a teacher's name requires multiple updates.

Insertion anomaly:

A new subject or teacher cannot be stored independently.

Deletion anomaly:

Deleting the last student of a class may remove class-related information.

Normalized Design:

STUDENT:

STUDENT(StudentID, StudentName, ClassID)

CLASS:

CLASS(ClassID, ClassName, TeacherID)

TEACHER:

TEACHER(TeacherID, TeacherName)

SUBJECT:

SUBJECT(SubjectID, SubjectName)

ENROLLMENT:

ENROLLMENT(StudentID, SubjectID, Grade)

Keys:

StudentID – Primary Key of STUDENT

ClassID – Primary Key of CLASS

TeacherID – Primary Key of TEACHER

SubjectID – Primary Key of SUBJECT

(StudentID, SubjectID) – Primary Key of ENROLLMENT

Foreign Keys:

ClassID in STUDENT references CLASS.

TeacherID in CLASS references TEACHER.

StudentID and SubjectID in ENROLLMENT reference STUDENT and SUBJECT.

BCNF Justification:

Each non-trivial functional dependency has a determinant that is a candidate key of its relation:

StudentID $\rightarrow$ StudentName, ClassID

ClassID $\rightarrow$ ClassName, TeacherID

TeacherID $\rightarrow$ TeacherName

SubjectID $\rightarrow$ SubjectName

(StudentID, SubjectID) $\rightarrow$ Grade

Therefore, the normalized relations satisfy BCNF under the stated dependencies.

Conclusion:

The original School ERP schema is decomposed into STUDENT, CLASS, TEACHER, SUBJECT and ENROLLMENT. This removes redundancy and insertion, deletion and update anomalies while maintaining data integrity.

Q10. Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.**Answer:**

Since the question does not specify the complete movie streaming schema, consider the following representative relation:

STREAMING(UserID, MovieID, DeviceID)

Assume that a user can watch multiple movies, can use multiple devices, and a movie can be available on multiple devices independently.

The relevant relationships can create a join dependency:

$STREAMING = \pi(UserID, MovieID) \bowtie \pi(UserID, DeviceID) \bowtie \pi(MovieID, DeviceID)$

Therefore, the original relation can be decomposed into three relations.

5NF Decomposition:

USER_MOVIE:

USER_MOVIE(UserID, MovieID)

USER_DEVICE:

USER_DEVICE(UserID, DeviceID)

MOVIE_DEVICE:

MOVIE_DEVICE(MovieID, DeviceID)

Example:

USER_MOVIE:

UserID | MovieID

U1 | M1

U1 | M2

USER_DEVICE:

UserID | DeviceID

U1 | D1

U1 | D2

MOVIE_DEVICE:

MovieID | DeviceID

M1 | D1

M1 | D2

M2 | D1

M2 | D2

Lossless-Join Verification:

The original STREAMING relation can be reconstructed by joining the three decomposed relations:

$USER_MOVIE \bowtie USER_DEVICE \bowtie MOVIE_DEVICE$

If the resulting tuples are exactly the valid tuples of the original relation and no spurious tuples are introduced, the decomposition is lossless.

The join dependency is represented as:

$(UserID, MovieID), (UserID, DeviceID), (MovieID, DeviceID)$

Conclusion:

The decomposition separates the independent relationships between users, movies and devices. The original relation can be reconstructed through the specified joins, making the decomposition lossless under the assumed join dependency. This provides a 5NF design and reduces redundancy.